

A MINI PROJECT REPORT ON
MEDICAL INSURANCE COST PREDICTION USING
PYTHON FLASK

Submitted by,

ARE THIRUPATHI	23J41A0504
BHUKYA KALYANI	23J41A0510
BEERAM DIVYA REDDY	23J41A0509
KALLAGUNTA SAI SRI HARSHA	23J41A0531

in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Dr. P Dayaker

Assistant Professor, Computer Science and Engineering



Department of

COMPUTER SCIENCE AND ENGINEERING

MALLA REDDY ENGINEERING COLLEGE

(An UGC Autonomous Institution, Approved by AICTE, New Delhi & Affiliated to JNTUH,
Hyderabad)

Maisammaguda, Secunderabad, Telangana, India 500100

APRIL -2026

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100



BONAFIDE CERTIFICATE

This is to certify that this major project work entitled “**MEDICAL INSURANCE COST PREDICTION USING PYTHON FLASK**”, submitted by **ARE THIRUPATHI (23J41A0504), BHUKYA KALYANI (23J41A0510), BEERAM DIVYA REDDY (23J41A0509), KALLAGUNTA SRI SAI HARSHA (23J41A0531)** to

Malla Reddy Engineering College affiliated to JNTUH, Hyderabad in partial fulfilment for the award of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** is a bonafide record of project work carried out under our supervision during the academic year 2025 – 2026 and that this work has not been submitted elsewhere for a degree.

SIGNATURE

DR. P.DAYKER

Assistant Professor,

Computer Science and Engineering

Malla Reddy Engineering College

Secunderabad, 500 100

SIGNATURE

DR. P. VENKATESHWARLU

Professor & HOD,

Computer Science and Engineering

Malla Reddy Engineering College

Secunderabad, 500 100

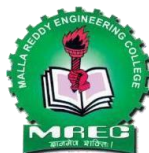
Submitted **for Mini Project** viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100



DECLARATION

I hereby declare that the project titled “**MEDICAL INSURANCE COST PREDICTION USING PYTHON FLASK**”, submitted to Malla Reddy Engineering College (Autonomous) and affiliated with JNTUH, Hyderabad, in partial fulfilment of the requirements for the award of a **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING**, represents my ideas in my own words. Wherever others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity, and I have not misrepresented, fabricated, or falsified any idea, data, fact, or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree

ARE THIRUPATHI

23J41A0504

Signature(s)

BHUKYA KALYANI

23J41A0510

BEERAM DIVYA REDDY

23J41A0509

KALLAGUNTA SAI SRI
HARSHA

23J41A0531

Secunderabad - 500 100

Date:

MALLA REDDY ENGINEERING COLLEGE

Maisammaguda, Secunderabad, Telangana, India 500100

ACKNOWLEDGEMENT

We would like to express our sincere thanks to **Dr. P.Ravinder Reddy, professor, Principal , MREC(A)** for providing the working facilities in the college.

Our sincere thanks and gratitude to **Dr.P.Venkateshwarlu, Professor & Head Department of CSE, MREC(A)** for all the timely support and valuable suggestions during the period of our project.

We are thankful to our Project Coordinator **Ms.Bodana.Gnaneswari, Assistant Professor**, Department of Computer Science and Engineering, for his cooperation during the project work.

We are greatly thankful and indebted to our internal guide, **Mr. P Dayaker, Assistant Professor, Department of CSE, MREC (A)** for his constant guidance, encouragement and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the CSE Department who helped us directly or indirectly, parents and friends for their cooperation in completing the project work.

ARE THIRUPATHI	23J41A0504
BHUKYA KALYANI	23J41A0510
BEERAM DIVYA REDDY	23J41A0509
KALLAGUNTA SAI SRI HARSHA	23J41A0531

ABSTRACT

The rising complexity of healthcare expenses has increased the demand for accurate medical insurance cost prediction systems. This project presents a machine learning-based approach to estimate individual insurance charges by analyzing demographic and health-related features such as age, gender, body mass index (BMI), number of dependents, smoking habits, and residential region. A publicly available dataset was preprocessed by encoding categorical attributes, scaling numerical variables, and visualizing relationships between predictors and charges. Multiple regression and ensemble algorithms, including Linear Regression, Support Vector Regression, Ridge Regression, and Random Forest Regressor, were implemented and evaluated. Comparative analysis revealed that the Random Forest Regressor outperformed other models, achieving an accuracy of approximately 86% in predicting insurance premiums. To enhance usability, the trained model was deployed using a Flask web application that provides real-time predictions based on user inputs. The system not only estimates the expected cost but also categorizes the user's BMI and assigns a risk level according to the predicted premium, thereby improving interpretability for end-users. Furthermore, predictions are logged systematically for monitoring and analysis, ensuring transparency of model behaviour. The integration of machine learning algorithms with a lightweight web framework demonstrates the feasibility of delivering practical and user-friendly solutions in the domain of health insurance analytics. This research highlights the significance of predictive modeling in fostering fair insurance pricing, assisting individual in financial planning, and aiding insurance providers in risk assessment.

KEY WORDS:

- Medical Insurance
- Cost Prediction
- Machine Learning
- Flask
- Regression

TABLE OF CONTENTS

ABSTRACT	V
LIST OF ABBREVIATIONS	VIII
LIST OF FIGURES	VIII

CHAPTER	DESCRIPTION	PAGE.NO
1	INTRODUCTION	1
1.1	INTRODUCTION	1
2	LITERATURE SURVEY	3
2.1	LITERATURE SURVEY	3
3	EXISTING SYSTEM	6
3.1	EXISTING SYSTEM	6
3.2	DISADVANTAGES	7
3.3	PROPOSED SYSTEM	7
3.4	ADVANTAGES	8
4	SYSTEM REQUIREMENTS	10
4.1	SYSTEM REQUIREMENTS	10
4.1.1	H/W SYSTEM CONFIGURATION	10
4.2	SOFTWARE REQUIREMENTS	10
5	SYSTEM ARCHITECTURE	11
5.1	SYSTEM ARCHITECTURE	11
6	SYSTEM STUDY	12
6.1	SYSTEM STUDY	12
6.1.1	FEASIBILITY SYSTEM	12
6.1.1.a	ECONOMICAL FEASIBILITY	12
6.1.1.b	TECHNICAL FEASIBILITY	12
6.1.1.c	SOCIAL FEASIBILITY	13
7	SYSTEM DESIGN	14
7.1	SYSTEM DESIGN	14
7.1.1	UML DIAGRAM	14
7.1.2	CLASS DIAGRAM	15
7.1.3	SEQUENCE DIAGRAM	16
7.1.4	ACTIVITY DIAGRAM	17
7.1.5	DATA FLOW DIAGRAM	17
7.1.6	COMPONENT DIAGRAM	18

CHAPTER	DESCRIPTION	PAGE.NO
7.1.7	USE CASE DIAGRAM	19
7.1.8	DEPLOYMENT DIAGRAM	20
7.1.9	ARCHITECTURE BLOCK DIAGRAM	21
7.2	SOFTWARE ENVIRONMENT	22
7.3	HISTORY OF PYTHON	27
7.4	MODULES USED IN PROJECT	28
8	SYSTEM TEST	30
8.1	SYSTEM TEST	30
8.2	TYPES OF TEST	30
8.3	IMPLEMENTATION	33
9	SCREEN SHOTS	35
10	CONCLUSION	42
10.1	CONCLUSION	42
10.2	FUTURE SCOPE	42
11	REFERENCES	43

LIST OF ABBREVIATIONS

MAE	Mean Absolute Error
RMSE	Root Mean squared Error
SVM	Support Vector Machine
UML	Unified Modeling Language
RF	Random Forest

LIST OF FIGURES

FIGURE NO	DESCRIPTION	PAGE NO
5.1	Architecture Diagram	11
7.1	Class Diagram	15
7.2	Sequence diagram	16
7.3	Activity Diagram	17
7.4	Data Flow Diagram	18
7.5	Component Diagram	18
7.6	Use Case Diagram	19
7.7	Deployment Diagram	20
7.8	Architectural Block Diagram	21
9	Output Screen	35-41

CHAPTER-1

1 INTRODUCTION

1.1 INTRODUCTION

Healthcare has become one of the most critical sectors worldwide, with rising medical expenses posing a challenge for both individuals and insurance providers. Predicting medical insurance costs is essential for ensuring fair premium pricing, effective financial planning, and efficient risk management. Traditional insurance cost estimation often relies on generalized statistical assumptions, which may not accurately reflect the unique health and demographic profiles of individuals.

This project focuses on developing a medical insurance cost prediction model using machine learning algorithms integrated with a Flask-based web application for real-time deployment. The dataset used in this research, obtained from a publicly available source, contains demographic and lifestyle-related variables such as age, gender, body mass index (BMI), smoking status, number of children, and residential region. These features have a direct influence on healthcare costs, making them suitable predictors for insurance premium estimation. Data preprocessing steps such as encoding categorical variables, handling skewness, and scaling were applied to prepare the dataset for model training. Visualization techniques, including heatmaps and feature distribution plots, were also used to explore dependencies between input variables and insurance charges.

Several machine learning models were implemented and compared, including Linear Regression, Support Vector Regression (SVR), Ridge Regression, and Random Forest Regressor. Among these, the Random Forest model achieved the best performance, reaching an accuracy of approximately 86% in predicting medical charges. Hyperparameter tuning was further employed to optimize the models and improve prediction reliability. The trained Random Forest model was then deployed using Flask, creating an interactive web interface that allows users to input their details and instantly receive estimated insurance costs. In addition to prediction, the system categorizes users based on BMI and assigns a risk level according to the predicted premium, thereby enhancing interpretability and transparency.

The integration of ML with a lightweight web framework like Flask demonstrates the practical feasibility of deploying predictive models in real-world applications. By logging predictions and generating interpretable outputs, the system ensures transparency and trustworthiness, which are vital in sensitive domains such as healthcare and insurance. This project highlights how predictive analytics can aid individuals in understanding potential medical expenses while assisting insurance providers in risk assessment and fair pricing strategies. Overall, the proposed system combines robust machine learning techniques with an accessible deployment platform to deliver a user-friendly, data-driven solution for medical insurance cost prediction.

This project presents a **Multi-Insurance Cost Prediction System** that leverages machine learning techniques to estimate premiums for multiple types of insurance policies. The system integrates predictions for the following insurance categories:

Medical Insurance Prediction – Based on health conditions, lifestyle, and demographic factors.

Car Insurance Prediction – Based on vehicle details, driving history, and risk factors.

Home Insurance Prediction – Based on property characteristics, location, and security measures.

Life Insurance Prediction – Based on age, health status, and lifestyle habits.

Travel Insurance Prediction – Based on trip details, destination risk, and coverage requirements.

CHAPTER-2

2 LITERATURE SURVEY

2.1 LITERATURE SURVEY

The insurance industry has experienced a significant transformation over the past few decades with the adoption of data-driven technologies such as machine learning and artificial intelligence. Traditionally, insurance premium calculation relied heavily on actuarial science, which uses statistical models and historical data to estimate risk. These traditional approaches, although reliable to some extent, are often limited by their dependence on predefined rules and assumptions. They lack the ability to capture complex, non-linear relationships among multiple risk factors and are not well-suited for handling large and diverse datasets. As a result, these methods often produce generalized predictions that may not accurately reflect individual risk profiles.

Traditional and Statistical Approaches

With the advancement of computational capabilities, statistical models such as linear regression and generalized linear models were introduced to improve insurance prediction. These models enhanced prediction accuracy compared to purely rule-based systems but still struggled to handle non-linear relationships and high-dimensional data. Although these methods provided better insights into risk assessment, they were still limited in their ability to adapt to dynamic and complex real-world scenarios, which led researchers to explore more advanced machine learning techniques.

Machine Learning in Insurance

Machine learning has emerged as a powerful approach for insurance cost prediction due to its ability to learn patterns from historical data and make accurate predictions. These models can handle large datasets and capture intricate relationships among variables without requiring explicit programming. Supervised learning techniques, particularly regression models, are widely used to estimate insurance premiums, while classification models are applied to categorize policyholders into risk groups. Compared to traditional methods, machine learning models provide improved accuracy, scalability, and automation, making them suitable for modern insurance systems.

Medical Insurance Prediction

In the domain of medical insurance, several studies have applied machine learning algorithms such as linear regression, decision trees, and random forest to predict healthcare costs based on factors like age, body mass index, smoking habits, and pre-existing medical conditions. Research findings consistently indicate that lifestyle factors, especially smoking and obesity, play a significant role in determining insurance costs. While linear regression models offer simplicity and interpretability, they often fail to capture complex patterns in data. Ensemble methods such as random forest have demonstrated superior performance by improving accuracy and reducing overfitting.

Car Insurance Prediction

In automobile insurance prediction, machine learning models are used to analyze driver behavior, vehicle characteristics, and accident history to estimate premiums. Studies show that variables such as driving experience, number of accidents, and vehicle type significantly influence insurance costs. Recent advancements have incorporated telematics data, which captures real-time driving behavior, to enhance prediction accuracy. However, many models still rely on static datasets and face challenges in modeling dynamic behavioral patterns effectively.

Home Insurance Prediction

Home insurance prediction involves evaluating property-related risks using features such as property value, geographic location, construction materials, and security systems. Research indicates that environmental factors, including flood risk and crime rates, have a major impact on insurance premiums. Some studies have incorporated geospatial data to improve prediction accuracy, but challenges remain in modeling environmental changes and adapting to evolving risk conditions.

Life Insurance Prediction

Life insurance prediction focuses on long-term risk assessment by analyzing factors such as age, health condition, lifestyle habits, and occupation. Machine learning models, including random forest and gradient boosting techniques, have shown better performance compared to traditional actuarial methods. These models can capture complex relationships between variables and provide personalized premium estimates. However, concerns related to data privacy, ethical considerations, and fairness in decision-making remain significant challenges in this domain.

Travel Insurance Prediction

Travel insurance prediction is a relatively emerging area that considers factors such as trip duration, destination risk level, travel purpose, and medical conditions. Studies suggest that destination risk and trip duration are among the most influential factors affecting premiums. Although machine learning models have improved prediction accuracy, the lack of comprehensive datasets and the variability of travel-related risks pose challenges for developing highly reliable systems.

Comparison of Algorithms

Various machine learning algorithms have been evaluated in the literature for insurance prediction. Linear regression is widely used for its simplicity but is limited in handling complex data patterns. Decision trees provide better interpretability but are prone to overfitting. Ensemble methods such as random forest and gradient boosting have emerged as powerful techniques due to their ability to improve accuracy and robustness. Neural networks, while capable of modeling highly complex relationships, require large datasets and high computational resources, which may limit their practical implementation.

CHAPTER-3

3 EXISTING AND PROPOSED SYSTEM

3.1 EXISTING SYSTEM

The existing systems used in the insurance industry primarily rely on traditional actuarial methods and rule-based approaches for premium calculation. These systems are designed based on predefined statistical formulas and historical data analysis, where actuaries evaluate risk factors such as age, income, health conditions, and asset details to determine insurance costs. In most cases, the process involves manual intervention, where experts analyze data and assign premiums based on standard guidelines and policies.

In recent years, some organizations have started adopting basic statistical models such as linear regression and generalized linear models to improve prediction accuracy. However, these models are still limited in their ability to capture complex relationships between multiple variables. Additionally, most existing systems are domain-specific, meaning they are designed to handle only one type of insurance, such as medical or automobile insurance, without integrating multiple domains into a unified platform.

Another important characteristic of existing systems is their lack of real-time prediction capabilities. Many systems require batch processing of data, where predictions are generated periodically rather than instantly. This reduces efficiency and delays decision-making. Furthermore, user interaction is often limited, as these systems are not designed with modern web interfaces, making them less accessible and less user-friendly for customers.

Overall, the existing systems are functional but lack flexibility, scalability, and intelligence. They are not well-equipped to handle large datasets or provide personalized insurance pricing, which has become increasingly important in today's data-driven environment.

3.2 DISADVANTAGES

Dependence on Manual Processes :

The existing system relies heavily on human intervention for calculating insurance premiums. Actuaries and experts manually analyze data and apply predefined rules, which makes the process time-consuming and increases the chances of human errors.

Low Prediction Accuracy:

Traditional methods use simple statistical models that cannot effectively capture complex relationships between multiple variables. As a result, the predictions are often generalized and may not accurately reflect individual risk profiles.

Lack of Personalization:

Existing systems provide standard premium calculations based on broad categories rather than individual-specific data. This leads to unfair pricing where customers with different risk levels may receive similar premium estimates.

Inability to Handle Complex Data:

Insurance data often involves multiple factors with non-linear relationships. Traditional models struggle to process such complex data, limiting their effectiveness in real-world scenarios.

No Real-Time Prediction:

Most existing systems operate on batch processing, where data is processed at intervals rather than instantly. This delays the prediction process and reduces system efficiency.

Limited Scalability:

Traditional systems are not designed to handle large volumes of data efficiently. As the amount of data increases, performance issues arise, making it difficult to scale the system.

3.3 PROPOSED SYSTEM

The proposed system introduces a Multi-Insurance Cost Prediction System that leverages machine learning techniques to overcome the limitations of existing systems. This system is designed to provide accurate, real-time predictions for multiple types of insurance, including medical, car, home, life, and travel insurance, within a single unified platform.

Unlike traditional systems, the proposed solution utilizes advanced machine learning algorithms, specifically the Random Forest Regression model, to analyze complex datasets and identify patterns among various risk factors. The model is trained using structured datasets that include

multiple features relevant to each insurance domain, enabling it to generate highly accurate predictions.

The system is implemented using the Flask web framework, which serves as the backend for handling user requests and delivering predictions. A user-friendly web interface allows users to input relevant details such as age, health conditions, vehicle information, or travel plans, and receive instant premium estimates. The integration of machine learning with a web-based platform ensures that predictions are generated in real time, improving efficiency and user experience.

Another key feature of the proposed system is its ability to provide insights into the prediction process. The system highlights important factors influencing the premium, such as top contributing features, and provides evaluation metrics like Mean Absolute Error and R-squared score to indicate model performance. This enhances transparency and helps users better understand the results.

The proposed system is also scalable and flexible, as it can handle large datasets and can be easily extended to include additional insurance domains or advanced models in the future. By integrating multiple insurance types into a single platform, the system simplifies the process for users and provides a comprehensive solution for insurance cost prediction.

3.4 ADVANTAGES

High Prediction Accuracy:

The proposed system uses advanced machine learning algorithms such as Random Forest, which can capture complex relationships between multiple variables. Unlike traditional methods, it analyzes patterns in data more effectively, resulting in more accurate and reliable insurance premium predictions.

Automation of Premium Calculation:

The system eliminates the need for manual calculations and human intervention. Once the user provides input data, the model automatically processes the information and generates predictions, reducing errors and saving time.

Real-Time Prediction:

With the integration of the Flask web framework, the system provides instant results. Users can input their details and receive insurance premium estimates immediately, improving efficiency and user experience.

Multi-Insurance Support:

Unlike existing systems that focus on a single domain, this system integrates multiple insurance types such as medical, car, home, life, and travel. This unified approach makes it more convenient and comprehensive for users.

Scalability:

The system is designed to handle large datasets and can be easily extended to include additional insurance types or new features. This makes it suitable for future enhancements and real-world applications.

User-Friendly Interface:

The web-based interface developed using Flask provides an interactive and easy-to-use platform. Users with minimal technical knowledge can access the system and obtain predictions without difficulty.

Improved Decision-Making:

By providing accurate predictions and insights, the system helps both customers and insurance companies make better decisions regarding policy selection and pricing.

CHAPTER-4

4.1 SYSTEM REQUIREMENTS :

4.1.1 H/W System Configuration: -

- Processor - intel Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

4.2 SOFTWARE REQUIREMENTS :

- Operating system : Windows 10/11 Ultimate.
- Coding Language : Python.
- Framework : Flask.
- Libraries and Tools : NumPy, Pandas, Scikit-learn, and Joblib.

CHAPTER-5

5.1 SYSTEM ARCHITECTURE

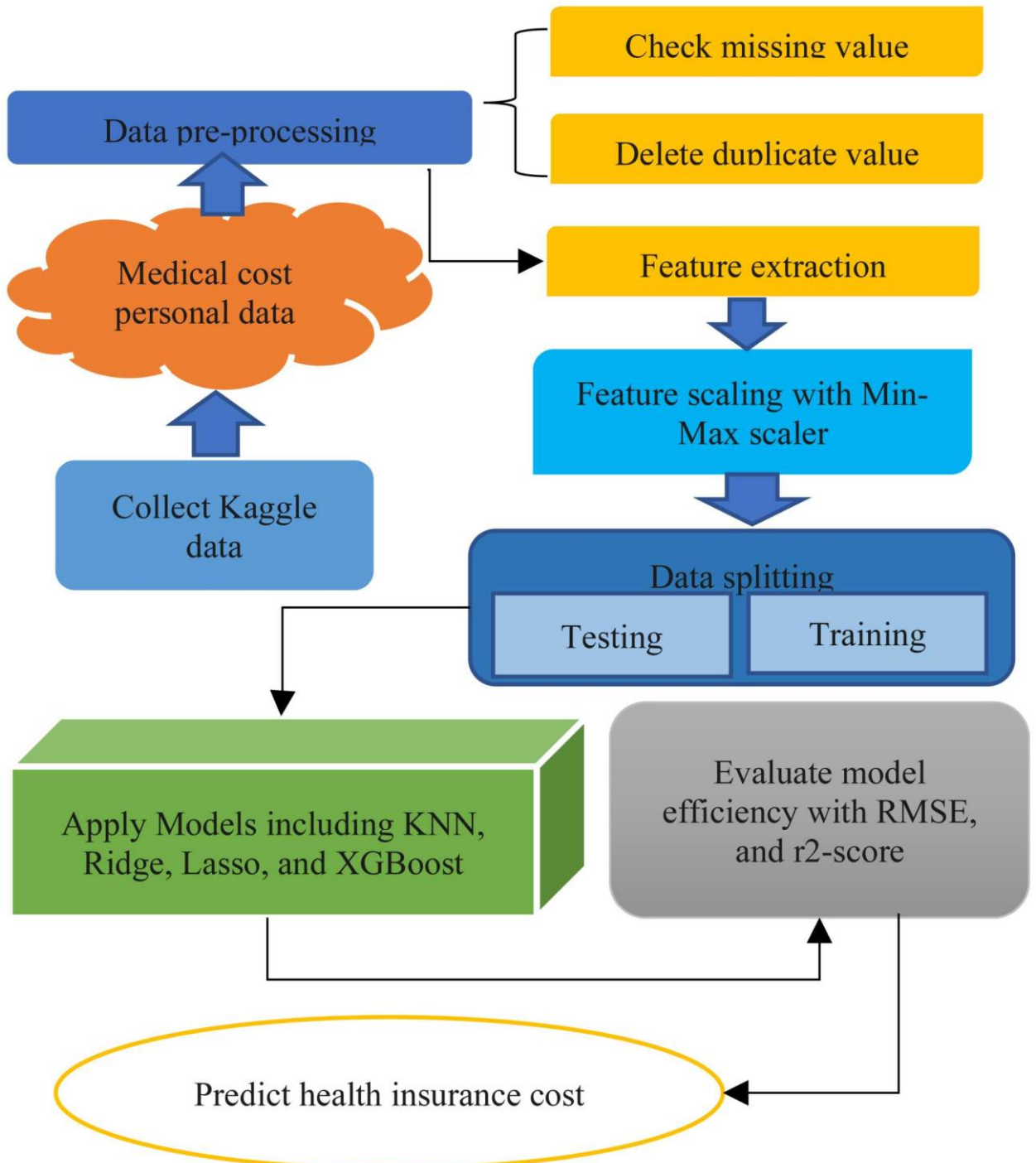


Fig-5.1:Architecture Diagram

CHAPTER-6

6.1 SYSTEM STUDY

6.1.1 FEASIBILITY STUDY

Feasibility study is a critical step in system analysis that determines whether the proposed system is viable and worth implementing. It evaluates the practicality of the system from various perspectives, including cost, technical resources, and user acceptance. The main objective of feasibility analysis is to ensure that the system does not impose unnecessary burdens and can be successfully developed and deployed.

Three key considerations involved in the feasibility analysis are

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

6.1.1.a ECONOMICAL FEASIBILITY

Economic feasibility refers to the cost-effectiveness of the proposed system. It evaluates whether the benefits of the system outweigh the costs involved in its development and implementation. In the case of the Multi-Insurance Cost Prediction System, the overall cost is relatively low because the project is developed using open-source tools and technologies such as Python, Flask, and Scikit-learn.

6.1.1.b TECHNICAL FEASIBILITY

Technical feasibility assesses whether the required technology and resources are available to develop and implement the system. The proposed system is built using widely used and well-supported technologies such as Python, Flask, and machine learning libraries. These technologies are easy to learn, highly reliable, and supported by large communities, making development and troubleshooting easier.

6.1.1.c SOCIAL FEASIBILITY

Social feasibility evaluates the acceptance of the system by users and its impact on society. The proposed system is designed to be user-friendly and accessible, ensuring that users with minimal technical knowledge can easily interact with it. The web-based interface allows users to input data and receive predictions in a simple and understandable format.

The system provides transparent results by highlighting key factors affecting insurance premiums, which helps build trust among users. It also promotes better decision-making by providing accurate and personalized predictions, benefiting both customers and insurance providers.

CHAPTER-7

7.1 SYSTEM DESIGN

7.1.1 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.

- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

7.1.2 Class diagram

The class diagram represents the static structure of the system by showing classes, their attributes, and methods. In this project, important classes include UserInput, DataProcessor, ModelHandler, and PredictionResult. The UserInput class collects data from the user, the DataProcessor prepares the data for prediction, the ModelHandler interacts with the machine learning model, and the PredictionResult class displays the output. The relationships between these classes help in understanding how data flows within the system.

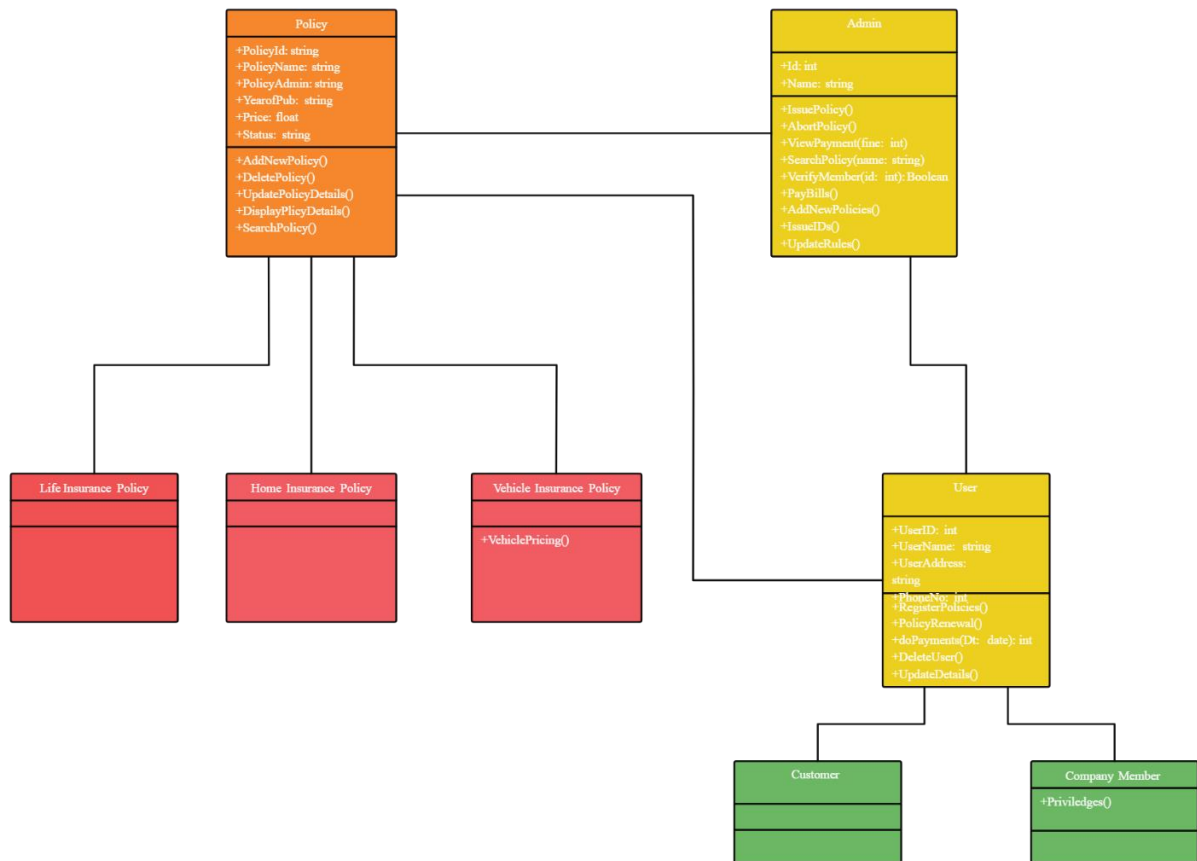


Figure-7.1: Class Diagram

7.1.3 Sequence Diagram

The sequence diagram shows the interaction between different components of the system in a time sequence. It explains how a user request is processed step by step. In this system, the user enters details and submits the form, the Flask server receives the request, processes the input, and sends it to the machine learning model. The model generates the prediction, and the result is returned to the user. This diagram helps in understanding the dynamic behavior of the system.

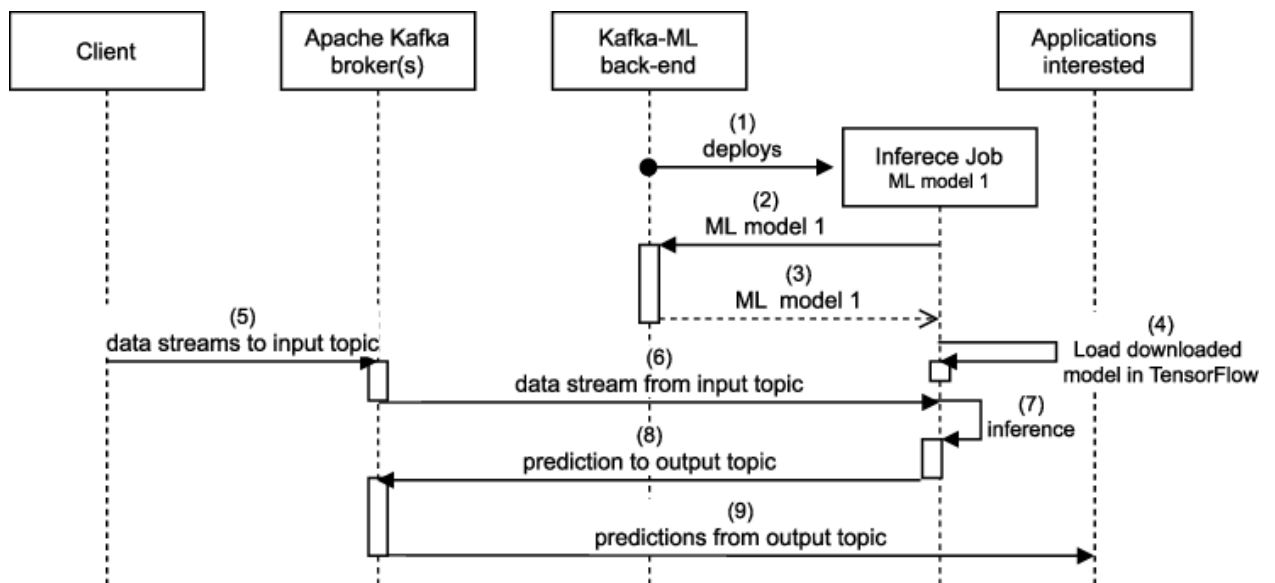


Figure-7.2: Sequence Diagram

7.1.4 Activity diagram

The activity diagram represents the workflow of the system. It shows the sequence of activities involved in the prediction process. The process begins with user input, followed by data preprocessing, model prediction, and result display. Decision points are included to handle different insurance types such as medical, car, home, life, and travel. This diagram helps in visualizing the logical flow of operations in the system.

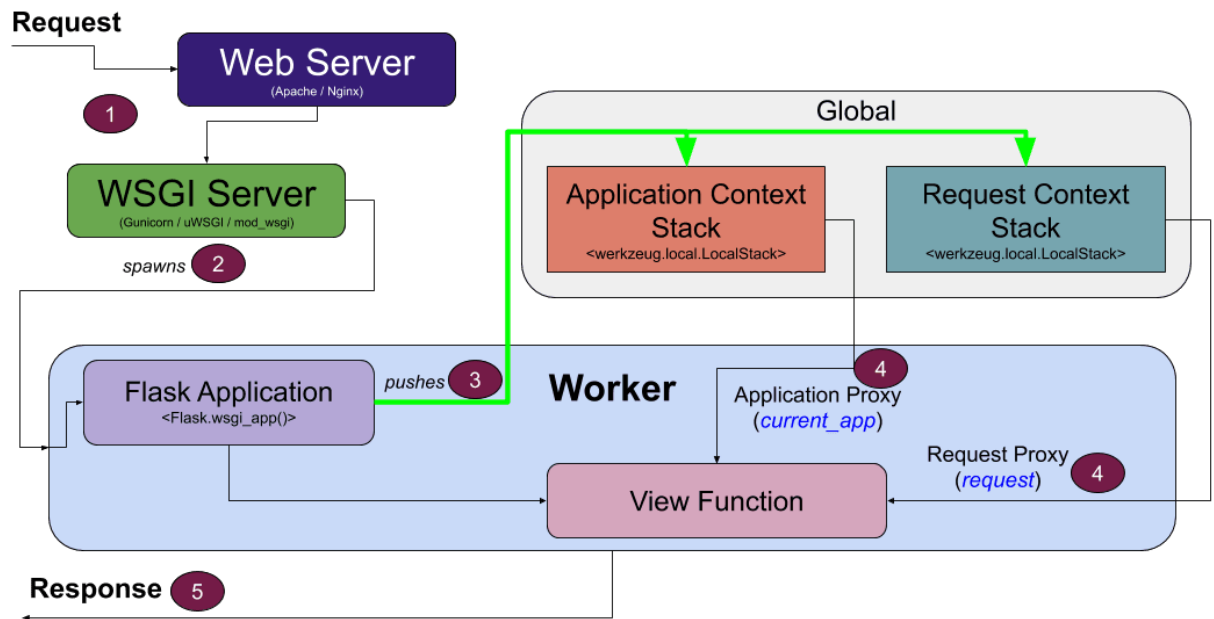


Figure-7.3: Activity Diagram

7.1.5 Data flow diagram

The Data Flow Diagram (DFD) shows how data moves through the system. It describes how input data is collected from the user, processed by the system, and transformed into output. The DFD includes processes such as data input, preprocessing, model prediction, and result generation. It helps in understanding how information is handled at each stage of the system.

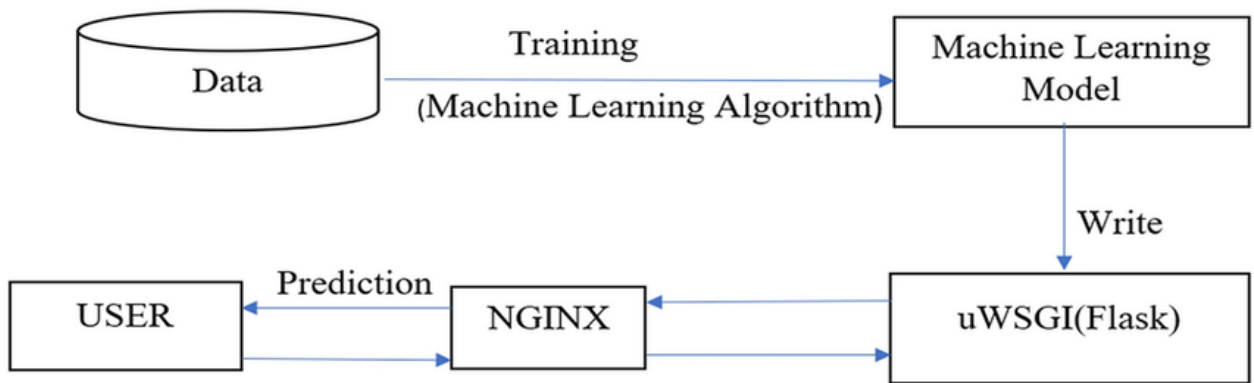


Figure-7.4: Dataflow Diagram

7.1.6 Component diagram:

The component diagram represents the different modules of the system and their interactions. The main components include the user interface, Flask backend, data preprocessing module, machine learning model, and dataset storage. The frontend collects user input, the backend processes the request, and the model generates predictions. This diagram shows how the system is divided into smaller components that work together.

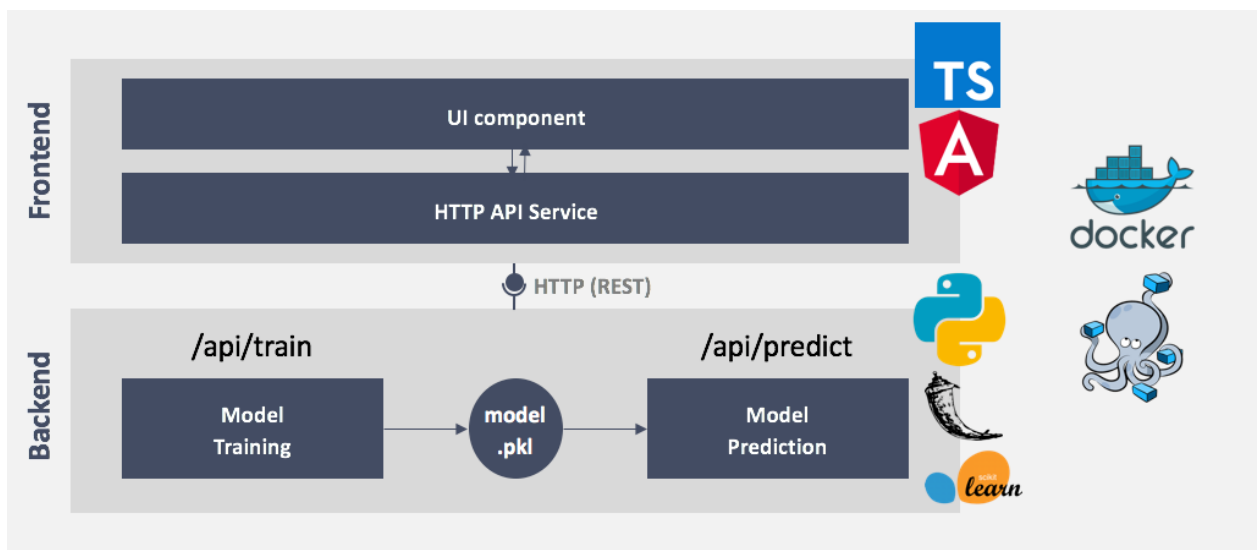


Figure-7.5: Component Diagram

7.1.7 Use Case diagram:

The use case diagram shows the interaction between the user and the system. The user is the main actor who performs actions such as entering details, selecting insurance type, and viewing results. The system processes these actions and provides predictions. This diagram helps in identifying the functional requirements of the system.

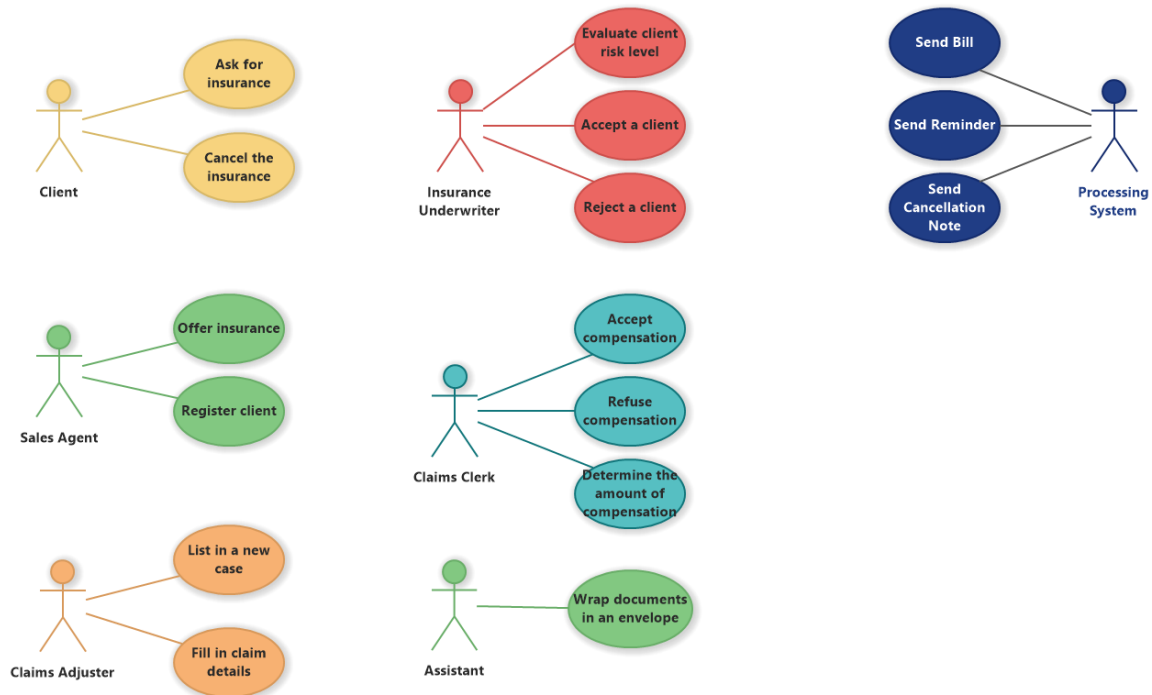


Figure-7.6: Use Case Diagram

7.1.8 Deployment Diagram:

The deployment diagram shows how the system is physically deployed. It includes components such as the client device (browser), server (Flask application), and machine learning model stored on the server. The user accesses the system through a browser, and the server processes the request and returns the result. This diagram helps in understanding the system's infrastructure.

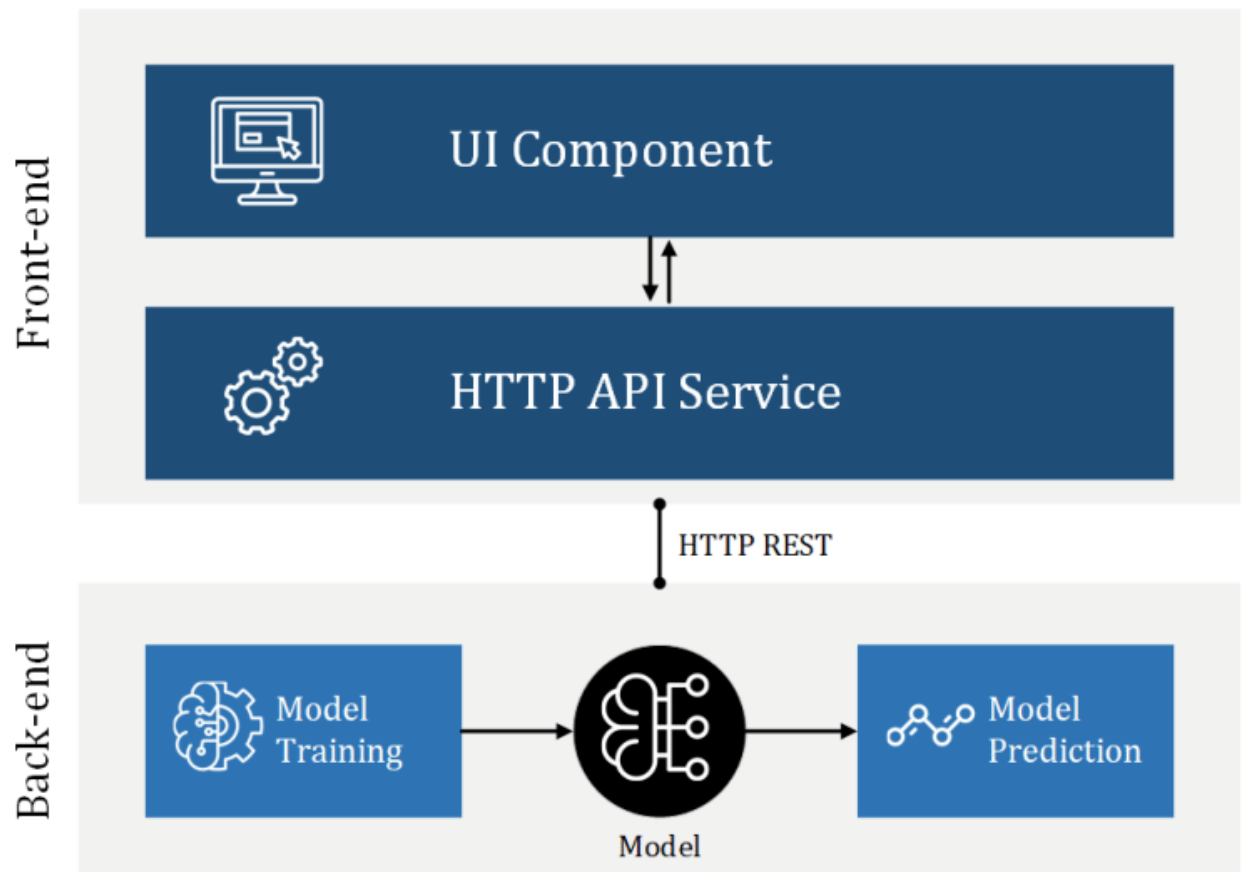


Figure-7.7: Deployment Diagram

7.1.9 Architectural Block Diagram

The architecture block diagram provides a high-level overview of the system. It shows the flow of data from input to output. The user enters data, which is processed by the backend, passed to the machine learning model, and the prediction result is displayed. This diagram helps in understanding the overall system structure.

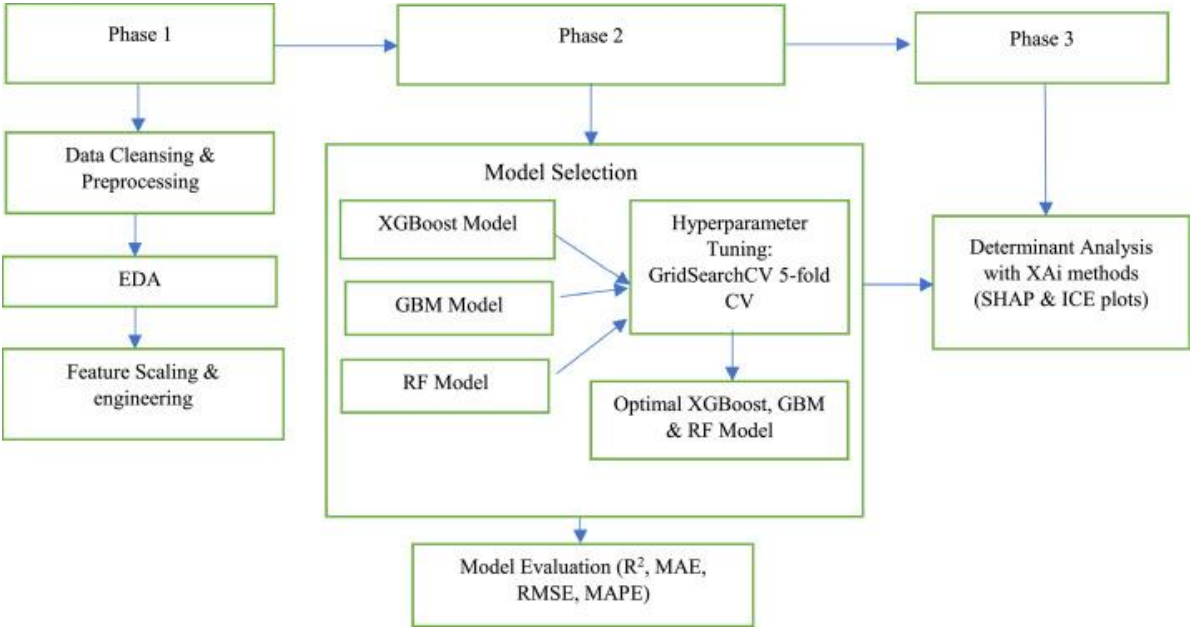


Figure-7.8: Architectural Block Diagram

7.2 SOFTWARE ENVIRONMENT:

What is Python:

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard libraries which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)

- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python:

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it *contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more*. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you [download Python](#) for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is

called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.\

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 GitHub annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and [machine learning](#), automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java Database Connectivity) and ODBC (Open Database Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

7.3 History of Python :

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So, I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

7.4 Modules Used in Project

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load,

prepare, manipulate, model, and analyse. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

CHAPTER-8

8.1 SYSTEM TEST

System testing is an important phase in software development used to verify that the entire system works correctly according to the requirements. In this project, system testing is performed to ensure that the Multi-Insurance Cost Prediction System provides accurate predictions and functions properly in different scenarios.

The testing process includes validating user inputs, checking data preprocessing steps, verifying model predictions, and ensuring proper communication between the frontend and backend. The Flask application is tested by entering various input values for different insurance types such as medical, car, home, life, and travel insurance.

The system is also tested for edge cases, such as missing or incorrect inputs, to ensure that appropriate error messages are displayed. Performance testing is conducted to check whether the system responds quickly and efficiently. Overall, system testing ensures that the application is reliable, accurate, and user-friendly.

8.2 TYPES OF TESTS

Unit testing

Unit testing is used to test individual components of the system, such as data preprocessing functions and machine learning model predictions. Each module is tested separately to ensure correct functionality.

Integration testing

Integration tests checks whether different components of the system work together properly. In this project, it ensures smooth interaction between the frontend, Flask backend, and machine learning model.

Functional test

Functional testing ensures that the system performs all required operations, such as accepting user inputs, processing data, and generating predictions.

System Test

System testing evaluates the complete system as a whole. It verifies that all functionalities, including input handling, prediction, and output display, work correctly.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

8.3 IMPLIMENTATION:

MODULES

Dataset Preparation

In this stage, datasets for different insurance types such as medical, car, home, life, and travel are collected or generated. The dataset contains features like age, BMI, smoking status, vehicle details, and other relevant parameters. The data is then cleaned by removing missing values and converting categorical data into numerical format.

Preprocessing Module

The collected data is preprocessed before feeding it into the machine learning model. This includes encoding categorical variables, normalizing values, and selecting important features. Proper preprocessing ensures that the model receives clean and structured data for accurate prediction.

Model Training Module

1. Machine learning models such as Random Forest are trained using the prepared dataset. The dataset is divided into training and testing sets, and the model learns patterns from the training data. After training, the model is evaluated using metrics like accuracy, MAE, and R^2 score

Backend Development (Flask)

The backend of the system is developed using Flask. It handles user requests, processes input data, and communicates with the machine learning model. API routes are created to manage prediction requests.

Frontend Development

The frontend is designed using HTML, CSS, and JavaScript. It provides a user-friendly interface where users can enter their details and view prediction results easily.

Integration of ML Model with Flask

The trained machine learning model is integrated with the Flask application. When the user submits input data, the backend processes it and sends it to the model for prediction. The result is then returned to the frontend.

CHAPTER-9

9.1 SCREEN SHOTS:

Medical insurance cost prediction using python flask.the file is provided in figure given below. In this below figure we can see all the files releted to medical insurance and datasets also requirements.

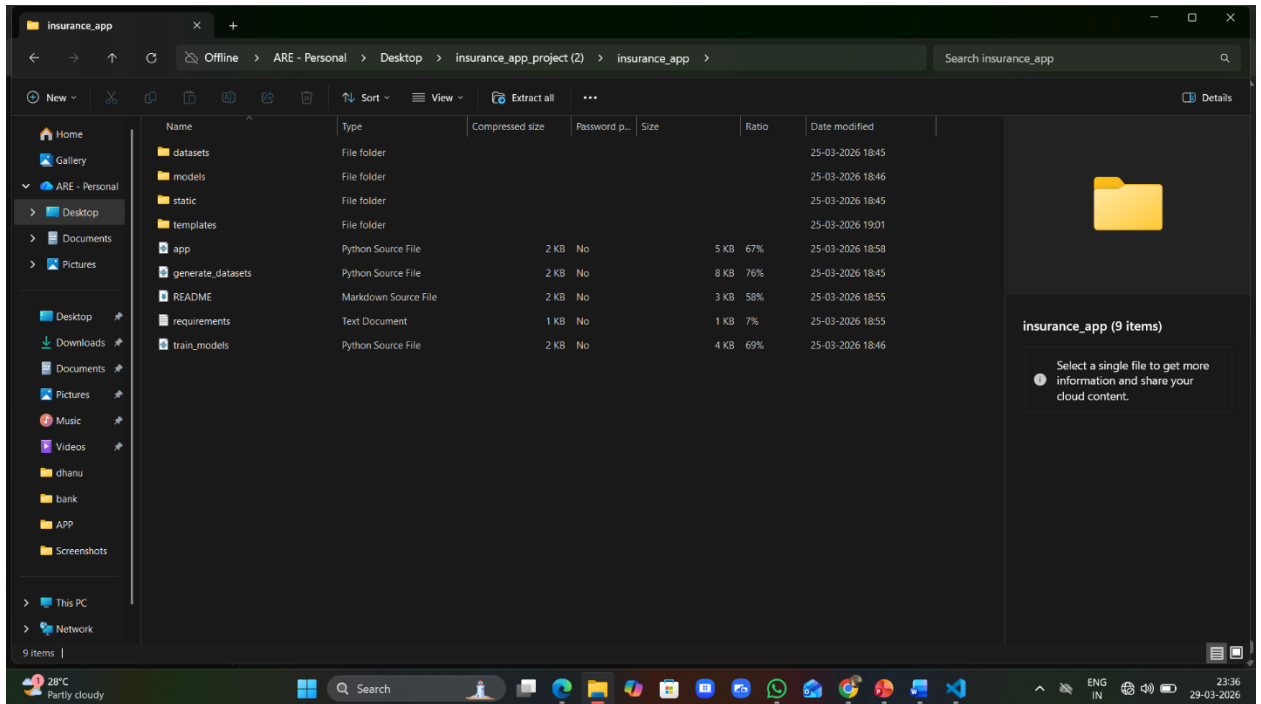


Figure-9.1: source code files

To implement this project we have designed following modules

1.Upload Insurance Dataset:

Using this module, we upload datasets related to different insurance types such as medical, car, home, life, and travel insurance. After uploading, the system loads the dataset and prepares it for further processing and model training.

2.Preprocess Dataset : In this module, the dataset is cleaned and prepared for machine learning. Missing values are handled, categorical data (like gender, smoker status, etc.) is converted into numerical form, and features are normalized. The dataset is then divided into training and testing sets, where 80% of the data is used for training and 20% for testing.

3.Train Model using Machine Learning Algorithm :

In this module, a machine learning model such as Random Forest is trained using the processed dataset. The model learns patterns from the training data and is then tested on the remaining data to evaluate its accuracy and performance.

4. Performance Evaluation Graph :

This module is used to analyze the performance of the trained model. Graphs such as accuracy, error rate, or feature importance are generated to visualize how well the model is performing.

5. Insurance Cost Prediction :

In this module, the user provides input details such as age, BMI, health condition, or other relevant factors. The trained model processes this input and predicts the insurance cost. The result is displayed along with additional insights if required.

6. Multi-Insurance Support Module :

This module allows users to select different types of insurance such as medical, car, home, life, or travel. Based on the selected type, the system uses the appropriate model to generate predictions.

7. User Interface Module :

This module provides a simple and user-friendly web interface using Flask. It allows users to enter data easily and view prediction results clearly.

SCREEN SHOTS

The Prediction Page is one of the most important components of the Multi-Insurance Cost Prediction System. It provides an interactive interface where users can enter their details and obtain estimated insurance costs in real time.

On this page, users are required to input relevant information based on the selected insurance type. For medical insurance, inputs may include age, gender, Body Mass Index (BMI), smoking status, and health conditions. Similarly, for other insurance types such as car, home, life, and travel, users provide specific details related to each category. The form is designed in a user-friendly manner, allowing easy data entry even for non-technical users.

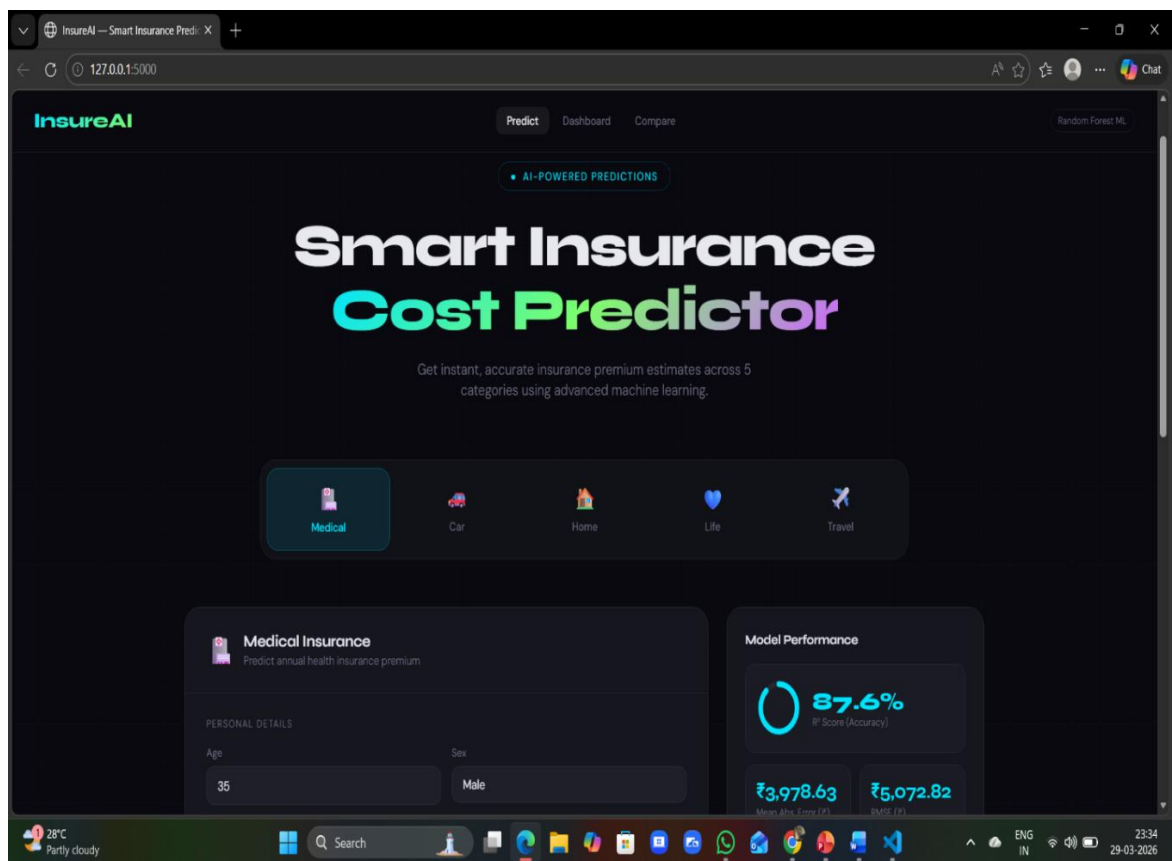


Figure-9.2: web application interface

The predicted result is then displayed on the same page or a result page. The output includes the estimated insurance premium and may also provide additional information such as risk level or influencing factors. Proper validation is implemented to ensure that incorrect or incomplete inputs are handled effectively by displaying appropriate error messages.

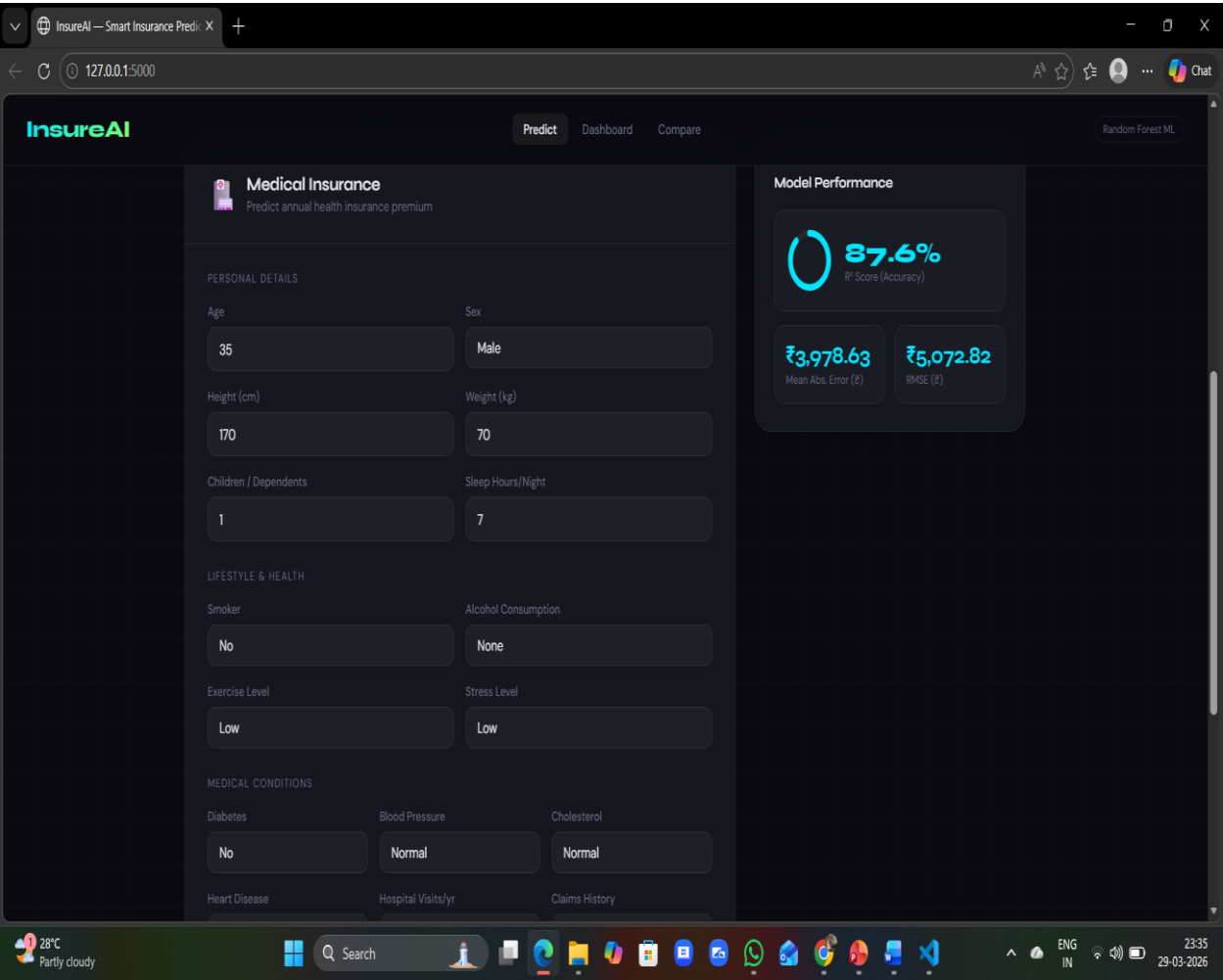


Figure-9.2: prediction page

The Dashboard Page serves as the main interface of the Multi-Insurance Cost Prediction System. It acts as the central hub where users can navigate through different functionalities of the application. The dashboard provides an overview of the system and allows users to access various insurance prediction modules such as medical, car, home, life, and travel insurance.



Figure-9.3: Dashboard page

The Comparison Page is an important feature of the Multi-Insurance Cost Prediction System that helps in analyzing and evaluating the performance of machine learning models used in the project. This page provides a visual representation of model performance using graphs and charts, making it easier to understand how well the model predicts insurance costs.

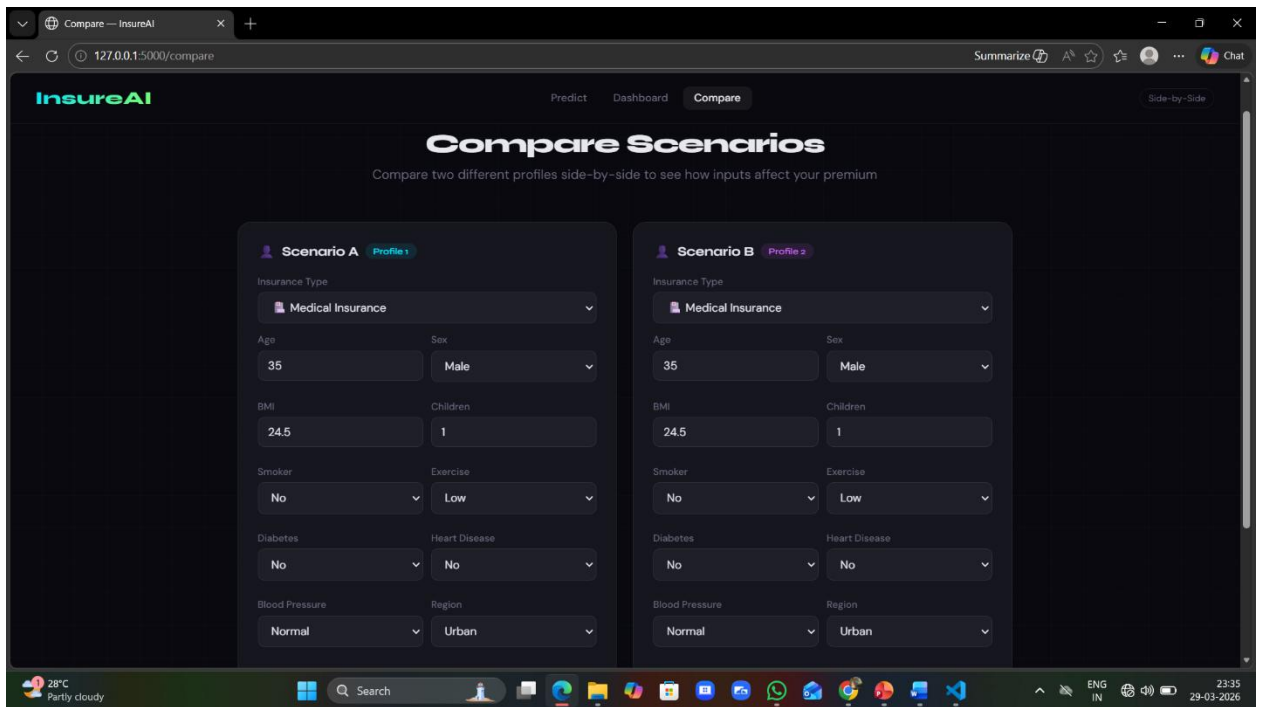


Figure-9.4: comparison page

The input section to predict the insurance premium with the suitable features.

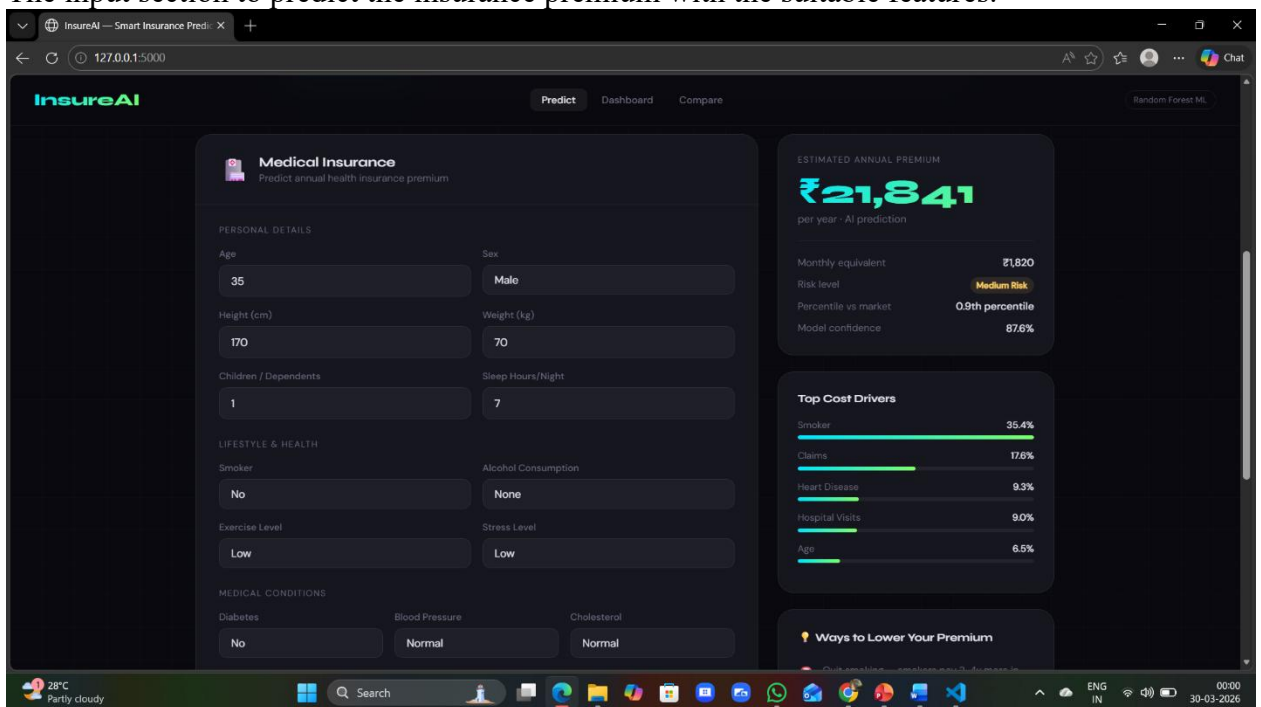


Figure-9.5: prediction input and result page

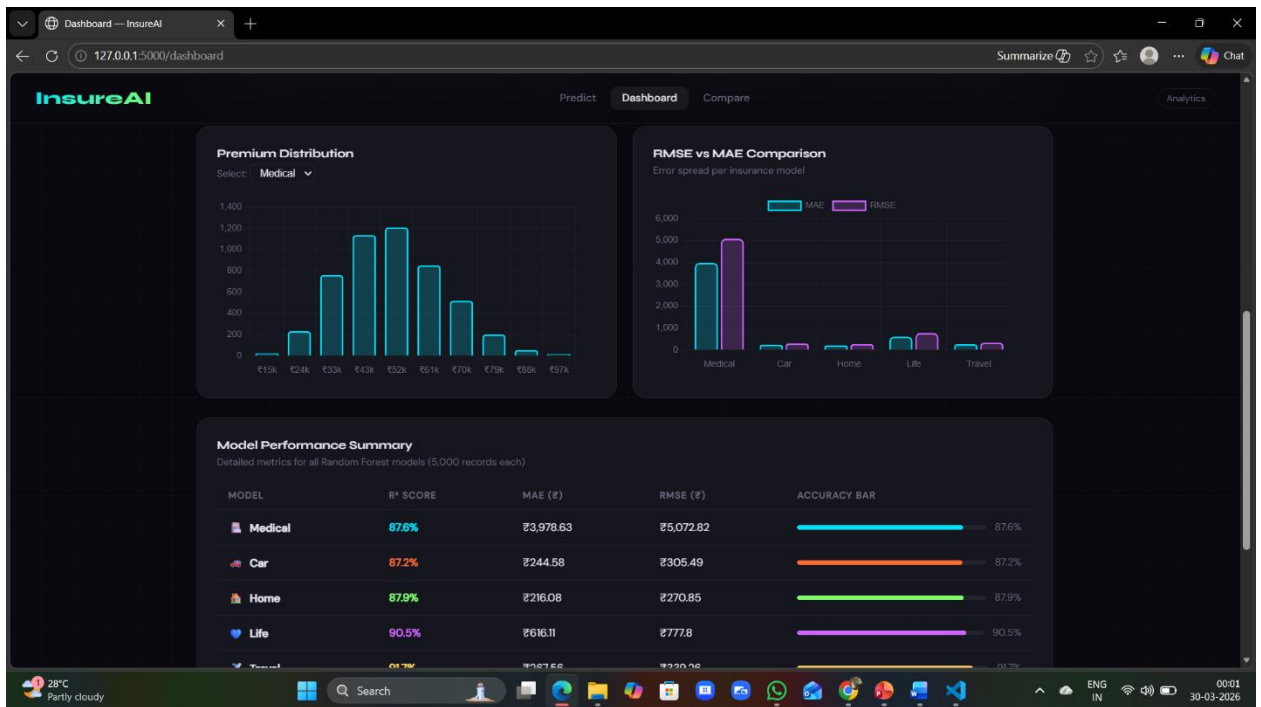


Figure-9.5: Dashboard result page

The comparison of two insurance types is given below. In this below figure we conclude that two insurance premiums can be compared to user by the given features.

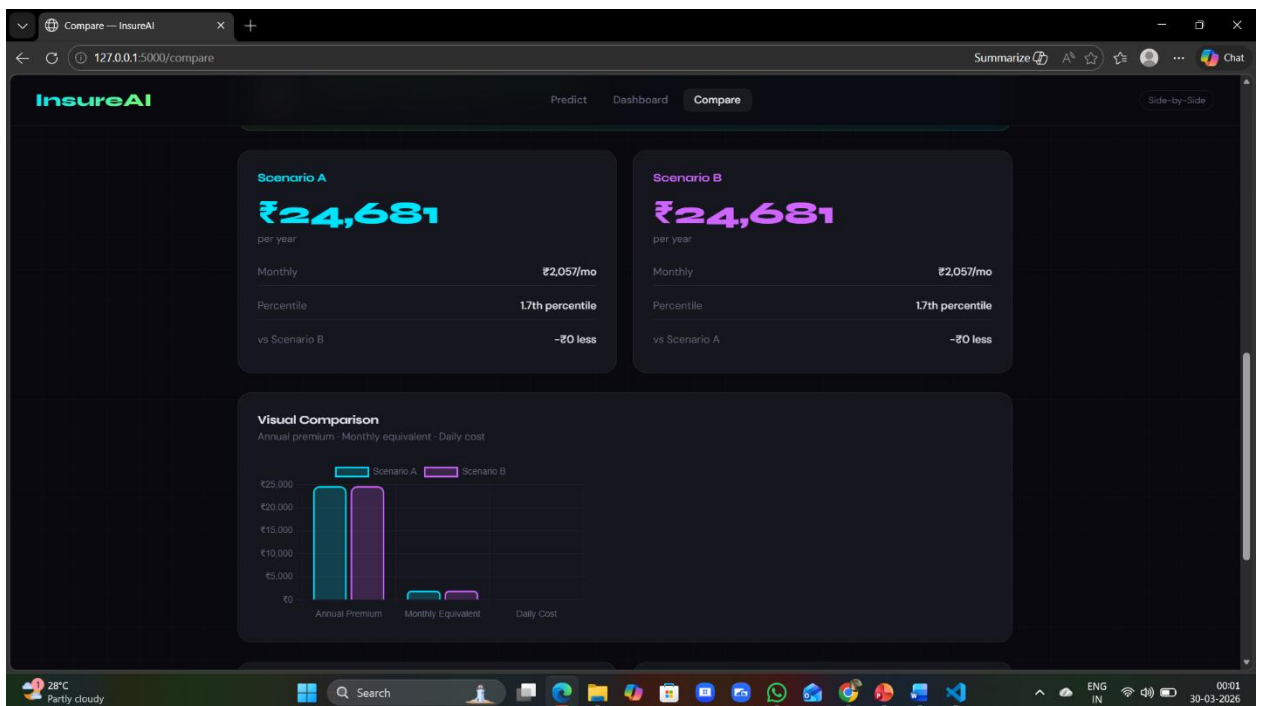


Figure-9.6: Comparison result page

CHAPTER-10

10.1 CONCLUSION:

The Medical-Insurance Cost Prediction System effectively demonstrates the application of machine learning techniques for accurate and efficient estimation of insurance premiums by integrating multiple insurance domains such as medical, car, home, life, and travel into a single platform, thereby improving convenience and usability. By utilizing algorithms like Random Forest and implementing the system through the Flask web framework, the application provides real-time predictions based on user inputs, significantly enhancing speed and accuracy compared to traditional methods. The system reduces manual effort, minimizes errors, and offers a user-friendly interface that allows users to easily input details and obtain instant results, while features such as performance comparison and data visualization improve transparency and understanding of the model. Overall, the project highlights the potential of machine learning in automating insurance prediction, supporting data-driven decision-making, and delivering an efficient, scalable solution suitable for both academic and real-world applications.

10.2 FUTURE SCOPE OF THE PROJECT:

The Multi-Insurance Cost Prediction System can be further enhanced by integrating real-time data from insurance companies and external APIs to improve prediction accuracy and reliability. Advanced machine learning techniques such as deep learning, gradient boosting, or hybrid models can be implemented to achieve better performance. The system can also be extended into a mobile application to increase accessibility for users. Deployment on cloud platforms like AWS or Azure can enable scalability and support multiple users simultaneously. Additionally, features such as personalized insurance recommendations, user account management, and historical data analysis can be incorporated to improve user experience. Enhancing data security and privacy measures will further strengthen the system. Overall, these improvements can transform the project into a robust, real-world solution for the insurance industry.

CHAPTER-11

11.1 REFERENCES

1. Breiman, L. (2001). "Random Forests." *Machine Learning*, 45(1), pp. 5–32.
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, pp. 2825–2830.
3. McKinney, W. (2010). "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, pp. 51–56.
4. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). "Array Programming with NumPy." *Nature*, 585, pp. 357–362.
5. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
6. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
7. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
8. Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly Media.
9. Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.
10. Kaggle. (2020). "Medical Insurance Cost Prediction Dataset." Available: <https://www.kaggle.com>
11. Python Software Foundation. (2023). "Python Documentation." Available: <https://www.python.org>